

问题教程：“蚁丘/蜜罐模拟器”

创意：Nikolai Durov 开发：Nikolai Durov

中心极限定理意味着，在 t 个单位时间后，蚂蚁的坐标 x 和 y 是独立的正态随机变量，均值为零，方差为 $t/2$ 。通过缩放，我们看到 $f(t)$ （蚂蚁在时间 t 处于蜜罐中的概率）是圆盘的高斯区域，其中心距原点距离为 $d' = d\sqrt{2/t}$ ，半径为 $R' = R\sqrt{2/t}$ 。它可以通过数值积分以多种不同的方式找到，例如，通过沿圆对微分形式 $\omega = \frac{1}{\sqrt{2\pi}}\Phi(y)e^{-x^2/2}dx$ 进行积分，其中 $\Phi(y)$ 是 $\frac{1}{\sqrt{2\pi}}e^{-y^2/2}dy$ 的任何反导数，例如 $\text{erf}(y/\sqrt{2})$ 。另一种选择是积分 $\omega' = \frac{1-e^{-(x^2+y^2)/2}}{2(x^2+y^2)}(x dy - y dx)$ ，它可以通过查看极坐标中的高斯密度来构造。沿着圆对解析函数进行积分是特别令人愉快的：只需计算圆上 2^k 均匀分布点上的函数值的平均值，然后让 k 增长，直到两个连续的近似值足够接近。一旦我们知道如何计算 $f(t)$ ，问题的答案就是 $(f(T), \int_0^T f(t) dt)$ 。

问题教程：“号码缺失”

创意：Vladislav Makarov 开发：Vladislav Makarov

乍一看，这个问题似乎不可能：拥有 4000 个状态可以被视为少于 12 位（因为 $2^{12} = 4096 > 4000$ ），并且直观上，需要这 12 位中的 3 位来以某种方式存储当前遍历数组的索引（因为 $8 = 2^3$ ），几乎没有任何内存可供我们使用。然而事实证明，4000 个状态已经足够了。

预期的解决方案基于二分搜索。在第一遍之前，我们知道数组中至少缺少一个 $[1, n+2)$ 范围内的整数。将 $[1, n+1]$ 划分为两个长度相等或几乎相等的范围 $[1, m)$ 和 $[m, n+2)$ 。在第一遍中，计算属于前一个范围的数组元素的计数。如果这样的元素少于 $m-1$ 个，则数组中缺少 $[1, m)$ 中的某个整数。另一方面，如果 $[1, m)$ 中的整数至少出现 $m-1$ 次（由于重复，出现次数可能多于 $m-1$ ），则 $[m, n+2)$ 中的整数最多出现 $n - (m-1) = n - m + 1 < (n+2) - m$ 次。

因此，在第一次运行之后，我们知道长度约为 $n/2$ 的范围，其中至少包含一个缺失的数字。类似地，如果我们当前知道 $[\ell, r)$ 范围内的整数丢失，我们可以选择 $m = \lfloor \frac{\ell+r}{2} \rfloor$ ，对 $[\ell, m)$ 范围内的整数进行计数，并根据结果得出结论， $[\ell, m)$ 或 $[m, r)$ 中的整数丢失。如果 $r - \ell = 2$ ，那么这两个范围都包含一个整数，这意味着我们知道答案。如果 $r - \ell \geq 3$ ，我们可能需要对数组进行更多次传递。很明显，这样的策略只需要 8 次传球。

事实上，在第一次传递之前，我们知道长度小于 2^8 的范围包含缺失的整数。在第一遍之后，我们知道长度最多为 2^7 的范围。在第二遍之后，我们知道长度最多为 2^6 的范围。依此类推，这意味着在第八遍之后，我们知道长度最多为 1 的范围，因此不需要更多的遍。

为什么这种方法能够满足状态数量的要求呢？好吧，一个完全幼稚的实现不会，但它很容易修复。假设我们当前的范围是 $[\ell, r)$ ，它分为长度几乎相等的范围 $[\ell, m)$ 和 $[m, r)$ 。然后，在对数组中出现的 $[\ell, m)$ 元素进行计数时，如果计数至少为 $m - \ell$ ，则超过 $m - \ell$ （是没有意义的，我们不关心其确切值，因为计数如此之高可确保 $[m, r)$ 的最多 $r - m - 1$ 个元素出现在数组中）。因此，我们的自动机的状态是二分搜索过程中可能出现的一段 $[\ell, r)$ 和一个从 0 到 $r - \ell$ （含）的整数。当然，当 $r - \ell = 1$ 时，我们不需要创建任何状态：每当我们想要创建导致这种状态的转换时，我们也可以立即停止并说 ℓ 是问题的正确答案。



事实证明，上述两种优化（不要计数太高，并且不要为只包含一个元素的范围创建任何状态）使得适应状态数量的限制变得非常容易：有一个范围要求我们从 0 计数到大约 $n/2$ （原始范围的 $n/2 + 1$ 状态），两个范围要求我们计数到大约 $n/4$ ，四个范围要求我们计数到大约 $n/8$ ，等等。因此，我们大约需要 $\sum_k 2^k \cdot (n/2^{k+1} + 1) = \sum_k 2^k \cdot (n/2^{k+1}) + \sum_k 2^k = 4n + 2^8 - 1 = 1255$ 个状态，远小于 4000 个（取决于“大约”一词的处理方式，可能比 1255 个状态多一点或少一点）在最终的自动机中，但仍然很明显，这种方法完全符合要求）。

问题教程：“弦乐研讨会”

创意：米哈伊尔·伊万诺夫 开发：
米哈伊尔·伊万诺夫

我们解释了“String Workshop”的预期 C++ 解决方案（声明中显示的代码）。整个方法依赖于两个观察：（1）可以稳定地选择最佳排序序列（交换相同的字母是无用的，只会增加费用），因此每个无序对必须恰好“交叉”一次；（2）交换费用取决于交换的相邻位置的当前索引 k ，因此我们必须计算所有强制交叉的加权和，而不仅仅是反转计数。

我们使用局部索引 $1..m$ 对子字符串进行排序。稳定的排序顺序是唯一确定的：所有“A”，然后所有“B”，……。。。，然后是所有“Z”，保留相同字母内的原始顺序。每个反转（一对 $p < q$ 与 $s[p] > s[q]$ ）在这两个具体事件变得相邻时，强制在它们之间进行一次相邻交换。然而，支付的价值是当时相邻交换的索引，这取决于元素在过程中如何移动。因此，我们需要一个可以聚合足够信息的数据结构来收回最小的总费用。

该解决方案在整个字符串 $s[1..n]$ 上构建一棵线段树。每个节点存储其段的类似幺半群的摘要：可以合并左子节点和右子节点的摘要以获得串联的摘要。这会在 $O(\log n)$ 中产生子字符串查询，并通过延迟传播的范围分配进行子字符串“突变排序”（类型 3）。

对于一个长度为 ℓ 的片段，对于每个字母 $c \in \{0, \dots, 25\}$ 商店

$$cnt[c] = \#\{p : s[p] = c\}, \quad sumPos[c] = \sum_{p: s[p]=c} p,$$

其中位置 p 是该段的本地位置（从 1 到 ℓ ）。这些数组让我们将右子元素中的所有位置移动左长度，并快速计算字母范围内的位置总和。

它还存储反转计数

$$inv = \#\{(p, q) : 1 \leq p < q \leq \ell, s[p] > s[q]\},$$

以及关键数量

$cost$ = 稳定排序该段所需的交换索引的最小可能总和。

为了计算 $O(26)$ 中两半之间的交叉贡献，节点另外存储字母阈值 b ,

$$pairCnt[b] = \#\{(p, q) : p < q, s[p] > b, s[q] \leq b\},$$

和

$$preCnt[b] = \sum_{q: s[q]=b} \#\{p < q : s[p] \leq b\}.$$

直观地说，在稳定合并步骤中，加权费用的公式涉及“左侧有多少个元素 $\leq b$ ”和“存在多少对形式 $(p, q) : p < q, s[p] > b, s[q] \leq b$ ”等术语，并且这两个数组精确地编码了这些计数。

现在考虑合并长度为 a 的左段 L 和长度为 b 的右段 R 。

1) 计数和位置总和:

$$cnt = cnt_L + cnt_R, \quad sumPos = sumPos_L + (sumPos_R + a \cdot cnt_R),$$

因为右半部分的每个位置都移动了 a 。

2) 反转:

$$inv = inv_L + inv_R + crossInv, \quad crossInv = \sum_{x \in R} \#\{y \in L : y > x\},$$

计算为 $\sum_{b=0}^{25} cnt_R[b] \cdot \#\{y \in L : y > b\}$ 。

3) 辅助数组与其定义相匹配:

$$pairCnt[b] = pairCnt_L[b] + pairCnt_R[b] + (\#\{y \in L : y > b\}) \cdot (\#\{x \in R : x \leq b\}),$$

$$preCnt[b] = preCnt_L[b] + preCnt_R[b] + cnt_R[b] \cdot (\#\{p \in L : s[p] \leq b\}).$$

4) 费用。我们可以将最佳稳定过程视为: 稳定排序 L , 稳定排序 R , 然后使用相邻交换稳定合并两个序列。在该合并中, R 中值为 b 的每个元素必须向左移动到 L 中值为 $> b$ 的所有元素; 每一次这样的移动都是一个相邻的掉期, 其费用等于当前掉期指数。将这些强制性费用相加即可得到 $cost$ 的叉部分。

该代码将合并费用表示为

$$cost = cost_L + cost_R + a \cdot inv_R + cross1 + cross2.$$

这里 $a \cdot inv_R$ 解释了右半部分的内部交换发生在移位 b 的索引处的事实
 a 在完整的段中。期限

y

$$cross1 = \sum_{b=0}^{25} cnt_R[b] \cdot \left(\sum_{y \in L: y > b} pos(y) \right)$$

当右侧元素经过左侧元素时, 使用 $sumPos$ 包含左侧元素贡献的位置相关部分。最后,

$$cross2 = \sum_{b=0}^{25} \left(cnt_R[b] \cdot pairCnt_L[b] + (\#\{y \in L : y > b\}) \cdot preCnt_R[b] \right)$$

捕获由正确元素移动并影响 eac 这一事实引起的剩余交互项

h

其他人的当前指数; 这正是节点维护 $pairCnt$ 和 $preCnt$ 的原因。

因此, 通过提取 $\mathcal{O}(\log n)$ 中 $[i, j]$ 的聚合并输出其 $cost$ 来回答子字符串查询 “2 i j”; 收集顺序是从左到右, 以便聚合内的局部索引根据需求为 $1..(j - i + 1)$ 。

更新和突变。

类型 1 是点分配。线段树支持惰性标签, 意思是“整个线段是一个字母”。应用这样的标记会设置 $inv = 0$ 和 $cost = 0$, 恰好设置 $1 \leq cnt[c] = \ell$, 并设置 $sumPos[c] = 1 + 2 + \dots + \ell$ 。因此, 改变一个点需要花费 $\mathcal{O}(26 \log n)$ 。

类型 3 首先查询 $[i, j]$ 并输出 $cost$, 然后用其排序版本替换子字符串。由于聚合已经包含计数 $cnt[0..25]$, 因此排序后的子字符串只是 26 个连续的统一块。代码赋值

$$[i, i + cnt[0] - 1] \leftarrow A, \quad [i + cnt[0], i + cnt[0] + cnt[1] - 1] \leftarrow B, \dots$$

使用惰性范围分配。这是 26 个赋值，每个 $\mathcal{O}(26 \log n)$ ，因此类型 3 是具有固定常量的 $\mathcal{O}(26^2 \log n)$ 。

每个测试的总体情况：在 $\mathcal{O}(26n)$ 中构建，在 $\mathcal{O}(26 \log n)$ 中输入 2，在 $\mathcal{O}(26 \log n)$ 中输入 1，在 $\mathcal{O}(26^2 \log n)$ 中输入 3。关键点是子串的答案直接存储在聚合幺半群的字段 *cost* 中。

问题教程：“可区分分布”

创意：米哈伊尔·伊万诺夫 开发：
米哈伊尔·伊万诺夫

解决这个问题的一种方法如下：取 $n = (t - 1) + 2^{t-1}$ 。我们将选择分布如下：我们将每个字符串 s 拆分为两个字符串 uv ，其中 $|u| = t - 1$ ， $|v| = 2^{t-1}$ ，如果 $v_u = \text{T}$ ，我们将调用 s 允许，其中下标中的 u 被理解为二进制数。我们的分布在允许的字符串上是均匀的（并且以 0 的概率允许不允许的字符串）。使用长度为 t 的 oracle 访问，可以很容易地检查任何字符串是否允许：前一个 $t - 1$ 查询将确定字符串 u ，最后，我们将找到符号 v_u 。事实上，这种分布是 $(2^{-t}t, t)$ 独立的，可以通过将仅符号 v_u 不同的所有字符串配对，考虑任何大小为 t 的索引集和这些索引处可能的二进制字符串的子集 T ，并检查该对中的一个字符串对应于 T ，而另一个不对应，这可以很容易地建立。完整的证明留给读者作为概率论的简单练习。

问题教程：“橡树周围的四圣人”

创意：米哈伊尔·伊万诺夫 开发：
米哈伊尔·伊万诺夫

将四位圣人的帽子涂成三种颜色总共有 $3^4 = 81$ 种方法，因此第一次运行中只有 81 种可能的输入。根据第一次运行的输入，可以构建第一次运行代码 - 一个从 0 到 80 的整数，唯一标识作为输入提供的颜色。例如，可以通过以下方式构建代码：将四顶帽子的颜色转换为集合 $\{0, 1, 2\}$ 中的数字，将它们视为数字的三进制数字（例如，右侧第 i 位数字对应于第 i 位圣人的帽子颜色），然后计算所得三进制数字（可能带有前导零）。例如，如果我们考虑 $\text{red} = 0$ 、 $\text{green} = 1$ 、 $\text{blue} = 2$ ，则输入示例“绿红蓝红”对应于数字序列 1、0、2、0，三进制数 0201_3 等于 13。

我们将第一个运行策略称为集合 $\{1, 2, 3, 4\}$ 中的 81 个数字的数组；那么我们可以在第一个运行策略 s_1 的情况下执行如下操作：根据输入数据计算代码 c_1 并以数字 $s_1[c_1]$ 进行响应。因此，首次运行策略与首次运行期间的所有 4^{81} 确定性方式完全对应。

类似地，第二次运行中有 $4 \cdot 3 \cdot 3 = 36$ 个可能的输入，可以将其编码为数字 0 到 35，所以第二次运行策略可以简单地认为是一个 36 个三进制数字的数组。 m

请注意，如果我们能够以某种方式获得某个正确解决方案的第二次运行策略，那么从中构建第一次运行策略并不困难。事实上，在第一次运行中，我们有四种响应方式：1, 2, 3, 4。我们可以尝试所有这些，看看在第二次运行中得到什么输入，以及根据第二次运行策略我们会给出什么答案。比如说，对于数字 3，如果我们在第二轮中给出的答案与第一次轮中第三位圣人的帽子颜色相匹配，那么我们可以在第一轮中回答 3。第一次运行时很可能有几个正确答案——我们可以选择其中任何一个。只有当四个答案都不合适时，问题才会出现。因此，如果第二次运行策略可以与第一次运行策略相补充以形成完整的解决方案，我们将称第二次运行策略是正确的。

因此，我们将问题简化为寻找正确的第二次运行策略。这是一个 36 个三进制数字的字符串（例如可以编码为 9 个字节），因此我们可以首先在本地找到它（不将其发送到测试系统，而是检查在工作计算机上第一次运行的所有 81 个输入），然后在实际提交中使用生成的数字数组，这将不再依赖于长时间的枚举。首先想到的是随机枚举这 36 个三进制数字，但是



这样的枚举可能需要几个小时。让我们行动得更聪明吧。我们将从字符串 s_2 开始，由 36 个三组成。三——是一个特殊的数字，意思是“我们不会在第二次运行时对此输入做出响应”。这样的策略肯定是不正确的，所以我们会开始逐步纠正它。让我们想象一下，并非第一次运行的所有输入都同样重要：有些输入是我们优先考虑的，并且我们肯定希望在这些输入上成功，而在其他不太重要的输入上，只要我们的解决方案在所有优先级更高的输入上都没有失败，就可能不会成功。形式上，我们将输入之间的这种关系表示为数字排列 $[0..80]$ ，我们将其表示为 $p \in \mathbb{S}_{[0..80]}$ ：第一次运行的代码出现在排列 p 中的时间越早，我们的解决方案在该代码上成功的优先级就越高。

修复 p ，让我们从左到右迭代它的代码并贪婪地执行操作。假设我们取了下一个代码 c_1 ；让我们看看如果我们用 1 响应会发生什么。这将为第二次运行 $c_2^{(1)}$ 产生一些输入。如果在 $c_2^{(1)}$ 上我们已经从 c_1 (输出了第一个圣人的帽子颜色，换句话说， $s_2[c_2^{(1)}] = c_1[1]$)，那么我们已经通过了测试 c_1 ，我们可以在 p 中继续。如果 $s_2[c_2^{(1)}] = 3$ ，那么我们还没有决定在第二次运行期间对这样的输入做什么。然后让我们分配 $s_2[c_2^{(1)}] := c_1[1]$ （现在解决方案通过了测试 c_1 ，我们刚刚保证了这一点！）并继续沿 p 前进。如果 $s_2[c_2^{(1)}]$ 已被分配并且包含除 $c_1[1]$ 以外的内容，那么我们将尝试用数字 2 响应 c_1 ，构造第二次运行的输入 $c_2^{(2)}$ - 并考虑相同的三种情况： $s_2[c_2^{(2)}] = 3$ 、 $s_2[c_2^{(2)}] = c_1[2]$ 、 $s_2[c_2^{(2)}] = c_1[3]$ ；在最后一种情况下，我们还必须尝试用数字 3 进行响应，最后用数字 4 进行响应。如果这四个选项都不起作用，那么策略 s_2 已经达到了无法补充为正确的状态，因此我们将排列 p 拒绝为坏排列。

现在我们可以按如下方式组织枚举：我们将在循环中随机选择 $p \in \mathbb{S}_{[0..80]}$ ，并查看是否可以按照上一段所述的操作获得正确的策略。这个方法已经见效比较快了，只要五分钟就可以在本地找到工作策略。

问题教程：“优点和缺点”

创意：Ivan Bochkov 开发：Ivan Bochkov

我们被要求计算总和 $\sum_a d_a \binom{2n-1-a}{n-1}$ ，其中 $d = 0, 1$ 取决于是否写成 i 米莎的笔记本。

让我们选择这样的系数 e_a 和 $\sum_a d_a \binom{2n-1-a}{n-1} = \sum_a e_a \binom{2n}{n+a}$ 。

请注意 $\binom{2n}{n+a} = \sum_k \binom{2n-k-1}{n-1} \binom{k}{a}$ 。从这里

$$\sum_a e_a \binom{2n}{n+a} = \sum_a e_a \sum_k \binom{2n-k-1}{n-1} \binom{k}{a} = \sum_k \binom{2n-k-1}{n-1} \sum_a \binom{k}{a} e_a$$

。

也就是说，如果 $d_k = \sum_{a=0}^k \binom{k}{a} e_a$ 对于任何 k ，则这组 e 是合适的。

但在这种情况下， $e_k = \sum_{a=0}^k \binom{k}{a} (-1)^{a-k} d_a$ 是合适的。

现在我们需要计算任意 n 的总和 $\sum_a e_a \binom{2n}{n+a}$ 。这与计算 $Q(x) = \sum e_a x^{-a}$ 和所有 n 的 $[x^0](x+1/x)^{2n} Q(x)$ 相同。我们使用分治算法来做到这一点。让我们在长度为 L 的线段上解决这个问题。然后，在 Q 中，我们只关心段 $[-L, L]$ 中的幂。将线段分成两半，并分别求解两半中的 $Q(x)$ 和 $Q(x)(x+1/x)^{2(L/2)}$ 问题。瓶颈在于将 $O(L \log L)$ 中长度为 L 的多项式相乘，这会导致总运行时间为 $O(n \log^2 n)$ 。

问题教程：“交通灯”

创意：米哈伊尔·伊万诺夫 开发：米哈伊尔·伊万诺夫

问题（算法对比 w）。我们有一棵顶点颜色为 $\{R, G, Y\}$ 的树 和两种查询类型：

- 类型 1: 重新着色顶点 v 。
- 类型 2: 对于给定的顶点 v ，找到一些红色 r 和绿色 g 之间的路径的最小长度，使得 v 位于路径 $r \leftrightarrow g$ 上。如果不可能，则输出 -1 。

关键重新制定。对于固定的 v 我们需要

$$\min_{\substack{r: \text{col}(r)=R \\ g: \text{col}(g)=G \\ v \in \text{path}(r,g)}} d(r, g).$$

如果 $v \in \text{path}(r, g)$ 那么 $d(r, g) = d(r, v) + d(v, g)$ ，所以我们希望两个端点在 v 处“相遇”。

LCA + 欧拉之旅。树的根位于 1。通过迭代 DFS 计算 $\text{tin}[u]$, $\text{tout}[u]$ 。那么 $\text{subtree}(u)$ 正是欧拉区间 $[\text{tin}[u], \text{tout}[u]]$ 。二元提升给出 LCA、距离和 kth_ancestor 。助手 $\text{next_neighbor}(v, x)$ 返回从 v 到 x 的第一步：如果 v 是 x 的祖先，则为路径上的子级，否则为 $\text{parent}[v]$ （如果 $x = v$ 则为 0）。这让我们可以通过单个禁止子树区间来描述“包含 x 的 v 的分支”。

质心分解（CD）。构建质心分解。对于每个顶点 u 存储其质心祖先链 $\text{cdAnc}[u] = [c_0, \dots, c_t]$ （在代码中从最近到最远迭代）。对于每个质心 c ，考虑其质心链包含 c 的质心分量的所有顶点。将它们的欧拉进入时间存储在排序数组 $\text{cenTin}[c]$ 中。

为什么要存储 $\text{cenTin}[c]$ ？任何有根子树约束都是欧拉区间 $[L, R]$ 。该子树与质心分量相交就成为 $\text{cenTin}[c]$ 中的连续索引范围（由 $\text{lower_bound}/\text{upper_bound}$ 找到）。

按质心的数据结构（对于每种颜色 $X \in \{R, G\}$ ）。

- 在 $\text{cenTin}[c]$ 索引上的线段树 $\text{segX}[c]$ ：如果 $\text{col}(x) = X$ ，则顶点 x 的叶子存储 $d(x, c)$ ，否则存储 ∞ 。因此范围最小值给出了允许的顶点之间的最小值 $d(x, c)$ 。
- 最小堆 $\text{heapX}[c]$ 存储对 $(d(x, c), x)$ ，以实现快速“无限制的最近”。它是惰性的：在查询时我们弹出，而存储的顶点不再是颜色 X 。

不受限制地最接近颜色 X 的 v 。对于每个质心 $c \in \text{cdAnc}[v]$ ：

$$\text{cand} = d(v, c) + \min_{x: \text{col}(x)=X} d(x, c),$$

其中最小值是从清理后的堆顶读取的。取所有 c 中最好的，同时返回节点 $\text{id}(\text{nearest_any_with_node})$ 。

最接近具有子树约束的颜色 X 的 v 。给定欧拉区间 $[L, R]$ ：对于每个 $c \in \text{cdAnc}[v]$ 计算 $\text{cenTin}[c]$ 中相应的索引范围 $[l_c, r_c]$ 。如果我们想要“内部” $[L, R]$ ，请在 $[l_c, r_c]$ 上查询 $\text{segX}[c]$ 。如果我们想要“外部”，则查询补码： $[0, l_c - 1]$ 和 $[r_c + 1, m - 1]$ 并取最小值。添加 $d(v, c)$ 并获取全局最小值。这是 $\text{nearest_interval_dist}$ 。

更新（类型 1）。重新着色 x 时：对于每个 $c \in \text{cdAnc}[x]$ 查找 $\text{cenTin}[c]$ 中 $\text{tin}[x]$ 的位置，将线段树中的旧颜色叶子设置为 ∞ ，将新颜色叶子设置为 $d(x, c)$ ，并将 $(d(x, c), x)$ 推送到新颜色堆（如果 R 或 G ）。

类型 2 查询逻辑（函数 min_traffic_light ）。令 (u, d_R) 为最接近 v 的无限制红色， (w, d_G) 为最接近绿色。还要处理简单的端点情况：如果 v 本身是 R （或 G ），我们可以将一个端点设置为 v 。

如果 v 位于路径 $u \leftrightarrow w$ 上，则 (u, w) 可行并给出候选项 $d_R + d_G$ 。



否则， u 和 w 位于 v 的同一分支中。让 $compR = next_neighbor(v, u)$ 和类似的 $compG$ 。任何可行对都必须在该分支之外至少有一个端点，否则路径永远不会到达 v 。所以我们尝试两种选择：

$$d_R + \min_{g \notin branch(compR)} d(v, g), \quad d_G + \min_{r \notin branch(compG)} d(v, r),$$

其中“在分支之外”被实现为“内部子树 (v)”（如果 $comp = parent[v]$ ）或“不在子树 ($comp$)”，否则（使用欧拉区间+线段树过滤）。在所有候选人中选出最好的。

复杂。CD 深度为 $O(\log n)$ 。对于每个质心，我们对 $cenTin[c]$ 和 $O(\log n)$ 线段树操作进行 $O(\log n)$ 二分搜索：

$$\text{Type 2: } O(\log^2 n), \quad \text{Type 1: } O(\log^2 n), \quad \text{memory: } O(n \log n).$$

问题教程：“甜蜜的剩余！”

创意：米哈伊尔·伊万诺夫 开发：

米哈伊尔·伊万诺夫

我们得到了多个测试用例。每个测试包含 n 、 m 和一个数组 $b_1, \dots, b_n$ 。如果 $b_i = -1$ ，则顶点 i 是空闲的。否则，我们必须有 $\deg(i) \bmod m = b_i$ 。我们需要输出“NO”或“YES”以及任何满足约束的树。

该解决方案分为两个独立的部分。首先，我们判断是否存在有效的度数序列，如果存在则构造一个。其次，我们构建一棵树，使用 Prüfer 代码构造来实现所选的度数。

$n > 1$ 个顶点上的树完全由具有 $d_i \geq 1$ 和 $\sum_i d_i = 2(n-1)$ 的度数序列 $d_1, \dots, d_n$ 来表征。因此，我们的目标是选择满足以下条件的整数 d_i ：

- $1 \leq d_i \leq n-1$ 对于所有 i （对于 $n=1$ ，唯一可能的次数是 0）；
- 对于每个受约束的顶点， $d_i \bmod m = b_i$ ；
- $\sum_i d_i = 2(n-1)$ 。

我们从最小的可行度开始。对于 $n=1$ ，答案是直接的：唯一的度数是 0，因此当且仅当 $b_1 = -1$ 或残差条件匹配零时，我们输出“YES”。

现在假设 $n > 1$ 。定义基度 $d_i^{(0)}$ ：

- 如果 $b_i = -1$ ，则设置 $d_i^{(0)} = 1$ ；
- 如果 $m = 1$ ，则所有残数必须为零，因此也设置 $d_i^{(0)} = 1$ ；
- 如果 $m > 1$ 和 $b_i \neq -1$ ，则选择具有所需残数的最小正整数： $d_i^{(0)} = b_i$ 表示 $b_i > 0$ ， $d_i^{(0)} = m$ 表示 $b_i = 0$ 。

如果任何 $d_i^{(0)} > n-1$ ，我们必须回答“否”。

令 $S_0 = \sum_i d_i^{(0)}$ 和 $R = 2(n-1) - S_0$ 为我们必须相加的剩余总和。如果 $R < 0$ ，则约束已经太严格，因此答案为“否”。

现在我们在不破坏残数且不超过 $n-1$ 的情况下分配 R ：

- 对于自由顶点，我们可以重复加 1 直到上限 $n-1 - d_i^{(0)}$ ；
- 对于受约束的顶点（当 $m > 1$ 时），我们只能添加 m 的倍数，同样达到上限。



这是最简单的两阶段检查。首先，我们使用单位步数在自由顶点上花费尽可能多的 R 。如果可用容量不足，则剩余部分必须以大小为 m 的步长被约束顶点覆盖。特别是，如果根本没有自由顶点且 $m > 1$ ，则 R 必须能被 m 整除，否则任何分布都无法保留所有残数。

一旦我们获得最终度数 d_i ，我们就使用 Prüfer 方法构造一棵具有这些度数的树。创建一个多重集，其中每个顶点 v 恰好包含 $d_v - 1$ 次，从而形成 Prüfer 序列。维护叶子的最小堆（当前度数为 1 的顶点）。从左到右处理 Prüfer 序列：弹出最小的叶子 u ，将其连接到下一个序列值 x ，递减度数，如果 x 成为叶子，则将其推入堆中。处理完所有 $n - 2$ 个符号后，剩下两个顶点；将它们与最后一条边连接起来。生成的树的构造度为 d_i ；因此它满足所有剩余约束。

每个测试用例的复杂度为 $\mathcal{O}(n)$ （用于构建度数）和 $\mathcal{O}(n \log n)$ （用于基于堆的 Prüfer 构建）。内存使用量为 $\mathcal{O}(n)$ 。

问题教程：“木制检查器”

创意：Vladislav Makarov 开发：Nikita Gaevoy

我们的任务是检查两个条件：森林的正确性和后代数量的正确性。我们将按顺序检查它们。

森林条件的正确性检查非常简单：足以验证每个顶点的入度不超过 1 并且图不包含（无向）循环。后一个条件可以使用不相交集系统来检查，而第一个条件可以使用简单的数组来验证。

为了检查后代数条件的正确性，我们需要进行一些简单的观察。首先，我们注意到，当添加边 $v \rightarrow u$ 时，后代集仅针对从 v 到包含 v 的树的根的路径上的顶点发生变化。仅此观察就足以使用链接切割树等数据结构来解决问题，但我们将进行另一个观察以获得更简单的解决方案。

考虑包含 v 和 u 的连通分量中的顶点集。这些集合必须形成段（因为它们是树根的后代集合），我们将其表示为 $[l_u, r_u]$ 和 $[l_v, r_v]$ 。如果这两个线段在它们的并集中没有形成一个线段，我们可以立即指出新合并的连通分量的根具有一组违反问题条件的邻居。在这种情况下，我们可以使用不相交集系统找到该顶点并立即输出错误消息。

否则，不失一般性，我们可以假设 $r_u + 1 = l_v$ ，因此，顶点 v 处满足条件的标准是顶点 l_v 从它的可达性。很容易看出，如果从 v 可达 l_v ，则不仅 v 满足条件，而且其所有祖先也满足条件。

因此，我们需要学习如何解决以下问题：对于给定的顶点 v ，判断顶点 l_v 和 r_v 是否位于顶点 v 的子树中。为了检查这个条件，我们可以显式存储每个连接组件从根到顶点 l_v 和 r_v 的路径。当合并两个组件时，我们可以通过从末尾删除元素直到遇到 v 来执行检查。此操作总体上适用于线性时间内的所有查询，因为删除的顶点不会重新出现在路径中。

重新计算合并组件的路径是使用单个串联操作来表达的。为了快速执行此操作，我们可以使用双向链表或双端队列，以从小到大的方式合并（这会将时间复杂度增加到 $\mathcal{O}(n \log n)$ ）。

最终时间复杂度估计为不相交集系统的时间复杂度，所需内存是线性的。

问题教程：“欧拉线”

创意：米哈伊尔·伊万诺夫 开发：米哈伊尔·伊万诺夫

关于欧拉线的数学事实

对于非简并三角形 ABC ，我们表示： O - 外心（垂直平分线的交点）， H - 垂心（高度的交点）， G - 质心（中线的交点）， O_9 - 九点中心。

从经典几何可知，点 O, G, H, O_9 位于同一条直线（欧拉线）上， G 以 $OG : GH = 1 : 2$ 的比例划分线段 OH ，而 O_9 是 OH 的中点。

一个方便的向量公式（从 O 开始计数）：存在等式

$$\overrightarrow{OH} = \overrightarrow{OA} + \overrightarrow{OB} + \overrightarrow{OC}.$$

然后自动

$$H = A + B + C, \quad G = \frac{A + B + C}{3}.$$

特别是， G 的坐标始终位于晶格 $\frac{1}{3}\mathbb{Z} \times \frac{1}{3}\mathbb{Z}$ 中。

存在的必要条件

输入提供一行

$$kx + \ell y + m = 0, \quad k^2 + \ell^2 > 0.$$

由于质心 G 必须位于欧拉直线上，因此也位于给定的直线上，我们有

$$kG_x + \ell G_y + m = 0.$$

乘以 3 并代入 $3G_x = A_x + B_x + C_x$, $3G_y = A_y + B_y + C_y$ ，我们得到

$$k(A_x + B_x + C_x) + \ell(A_y + B_y + C_y) + 3m = 0.$$

左边可以被 $g = \gcd(k, \ell)$ 整除，因此有必要

$$g \mid 3m.$$

将描述在该条件下始终产生有效三角形的构造，这意味着该条件也足够了。

为什么不能使用仿射变换“简单地将直线变换为垂直”

O 和 H 的定义主要依赖于垂直度（垂直平分线 and 高度）。任意整数线性/仿射变换不保留垂直性，因此它通常不会将 O 映射到图像的外心，将 H 映射到图像的正交中心。因此，为了在坐标系之间传递解，必须使用保留角度的变换，即相似性（旋转 + 缩放）、反射和平移。

关键思想：构造垂直欧拉线，然后应用相似性

考虑以下形式的整数矩阵

$$M = \begin{pmatrix} p & -q \\ q & p \end{pmatrix}.$$

这种变换是旋转和位似性（相似性）的组合：它保留了角度，从而保留了垂直性。因此，如果我们应用变换

$$(x, y) \mapsto (X, Y) = M \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix},$$

那么点 O, H, G, O_9 的图像将对应图像三角形的中心，欧拉线将变换为新三角形的欧拉线。

因此，只需：

1. 预先准备几个小整数三角形，其欧拉线是垂直的，形式为 $3x + r = 0$ ，其中 $r \in \{0, 1, 2\}$ ；
2. 对于给定的线，选择适当的 r 模 3；
3. 调整移位 (t_x, t_y) ，以便在应用 M 后，获得所需的行 $kx + \ell y + m = 0$ 。

从 (k, ℓ, m) 到相似性参数的转换

设 $g = \gcd(k, \ell)$ 和原始法线：

$$p = \frac{k}{g}, \quad q = \frac{\ell}{g}, \quad \gcd(p, q) = 1.$$

假设我们从一条垂直线开始

$$3x + r = 0 \iff x = -\frac{r}{3}.$$

我们使用矩阵 $M = \begin{pmatrix} p & -q \\ q & p \end{pmatrix}$ 和移位 (t_x, t_y) 进行变换。然后

$$X = px - qy + t_x, \quad Y = qx + py + t_y.$$

我们根据 (X, Y) 恢复 x ：

$$x = \frac{p(X - t_x) + q(Y - t_y)}{D}, \quad D = p^2 + q^2.$$

代入 $3x + r = 0$ ，可得垂直线图像方程：

$$3pX + 3qY + (Dr - 3(pt_x + qt_y)) = 0.$$

这是一条法线 $(3p, 3q)$ ，因此法线方向与原始线 (k, ℓ) 相同。

为了匹配常数，使自由项相等就足够了（在 $3/g$ 的范围内）：

$$Dr - 3(pt_x + qt_y) = M_0, \quad M_0 = \frac{3m}{g}.$$

这里，当且仅当满足条件 $g \mid 3m$ 时， M_0 才是整数。

选择余数 r 并求解移位方程

我们重写：

$$pt_x + qt_y = S, \quad S = \frac{Dr - M_0}{3}.$$

为了使 S 成为整数，我们需要实现比较

$$Dr \equiv M_0 \pmod{3}.$$

对于 $\gcd(p, q) = 1$ ， $p \equiv q \equiv 0 \pmod{3}$ 是不可能的，因此 $D = p^2 + q^2 \not\equiv 0 \pmod{3}$ 是可逆的模 3。因此，存在唯一满足 $Dr \equiv M_0 \pmod{3}$ 的 $r \in \{0, 1, 2\}$ ；因为这个 S 自动是一个整数。

此后，方程 $pt_x + qt_y = S$ 可以整数解，因为 $\gcd(p, q) = 1$ 。可以使用扩展欧几里德算法找到一个特定的移位。通用解决方案的形式为

$$t_x = t_x^{(0)} + q \cdot t, \quad t_y = t_y^{(0)} - p \cdot t, \quad t \in \mathbb{Z}.$$

这允许我们选择 t 来最小化值 $|t_x|, |t_y|$ （例如，使 t_x 接近 0 模 $|q|$ ），从而确保最终坐标不超过 $\pm 10^6$ 的限制。

底三角形

预先固定三个小整数非简并缩放三角形：对于 $r = 0$ ，欧拉线为 $3x + 0 = 0$ ，对于 $r = 1 - 3x + 1 = 0$ ，对于 $r = 2 - 3x + 2 = 0$ 。它们可以通过枚举经过精确验证的小坐标来获得（通过计算 O 和 H 并检查线 OH 的方程）。由于使用的变换是相似的，因此它保留了非简并性和（非）等腰属性：缩放三角形的图像将保持缩放状态。

最终算法

对于每个测试 (k, ℓ, m) ：

1. 计算 $g = \gcd(k, \ell)$ 。如果 $g \nmid 3m$ ，则输出六个零。 2. 设置 $p = k/g$ 、 $q = \ell/g$ 、 $D = p^2 + q^2$ 、 $M_0 = 3m/g$ 。
3. 从比较 $Dr \equiv M_0 \pmod{3}$ 中找到 $r \in \{0, 1, 2\}$ 。
4. 计算 $S = (Dr - M_0)/3$ 并找到方程 $pt_x + qt_y = S$ 的整数解 (t_x, t_y) 。 5. 在通解中选择参数 t 以最小化偏移并保持在坐标限制内。 6. 选取所选 r 的基本三角形并将变换应用于其顶点

$$(x, y) \mapsto (X, Y) = (px - qy + t_x, qx + py + t_y).$$

7. 输出结果的整数坐标。

为什么这有效

- 条件 $g \mid 3m$ 是必要的，因为质心始终具有分母为 3 的坐标。
- 当 $g \mid 3m$ 时，选择 r 可以实现自由项模 3 的精确对齐；那么移位可以精确地求解线性方程。
- 矩阵 M 定义了相似性，因此保留了直角和垂直度，这意味着 O, H, G, O_9 和欧拉线正确过渡到图像三角形中的相应对象。

问题教程：“三角形网格的遍历”

创意：Vladislav Makarov 开发：Vladislav Makarov

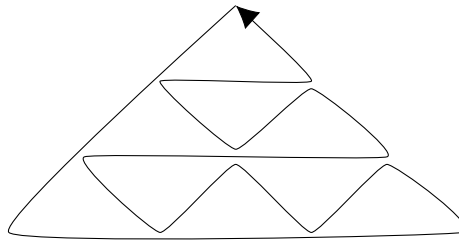
这是一个简单的问题，可以通过两种不同的方式来解决。首先，只要给定图中存在，就可以应用在给定图中找到欧拉回路的标准算法（例如，请参见 https://cp-algorithms.com/graph/euler_path.html 以了解该算法的一些说明）。这种方法非常简单，但需要相当多的时间来实施。

其次，该问题可以通过许多显式构造来解决。陪审团使用了以下一项：

1. 向左下走 n 步（即，直到碰到大三角形的左角）。 2. 向右走 n 步（直到碰到三角形的右角）。

3. 向左曲折移动 $2n - 1$ 步。具体来说，向左向上移动一步，然后向左向下移动一步，然后再次向左向上移动，等等。本节的最后一步是左上步骤，它将移动到三角形左侧第二低的网格顶点。4. 向右走 $n - 1$ 步（直到碰到三角形的右侧），然后以之字形方式向左移动 $2n - 3$ 步（直到再次碰到三角形的左侧）。5. 向右迈出一小步，重复相同的过程，直到到达三角形右侧第二高的顶点。之后，向左向上迈出一小步，完成欧拉回路。

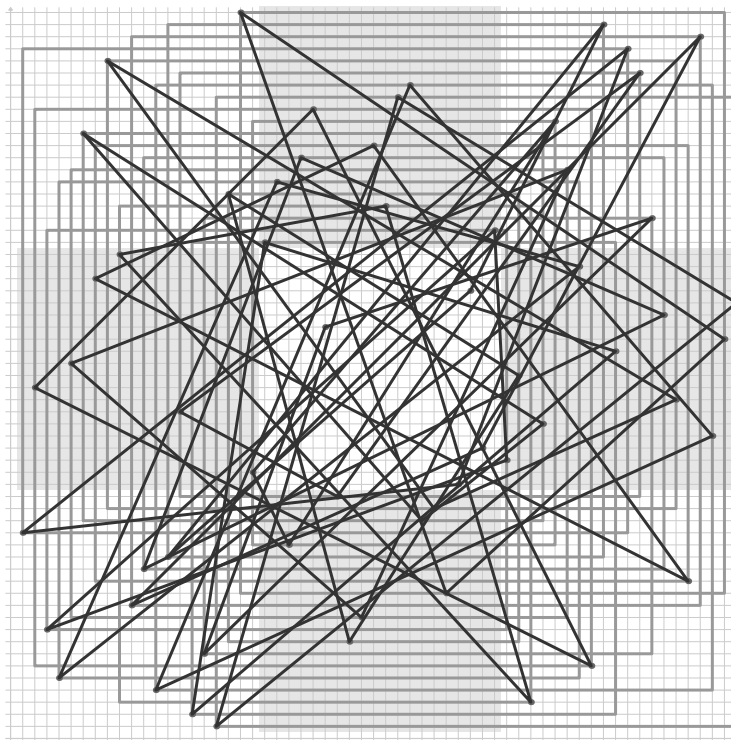
下图描述了当 $n = 3$ 时上述欧拉电路的样子：



问题教程：“三角形”

创意：Ivan Kazmenko 开发：Ivan Kazmenko

在由 3×3 个大小为 $(n/3) \times (n/3)$ 的单元格组成的点上画一个板。我们的每个点都位于其中一个单元格中。为了最大化答案，每个三角形必须包含棋盘左列的一个点、右列的一个点、顶行的一个点和底行的一个点。



对于每个三角形，查看这些点所属单元的坐标。我们希望它们是 $(1, p_1)$ 、 $(2, p_2)$ 和 $(3, p_3)$ ，其中 p 是 $(1, 2, 3)$ 的排列。我们将选择任何此类排列，并继续解决单元大小为 $(n/3 - 1) \times (n/3 - 1)$ 的子问题。

通过案例分析或流程可以证明，对于至少一个排列，所选的三个单元格中的每一个都包含一个点。

问题教程：“建筑公司”

创意：Ivan Bochkov 开发：Ivan
Bochkov

考虑一个图，其顶点 $0, 1, \dots, 2m$ (对应于时隙) 的末端。我们稍后将讨论带宽。

对每个 i 在顶点 i 和 $i + 1$ 之间画一条边，如果有任务 $[i, j]$ ，就画一条边 $(i - 1) \rightarrow j$ 。我们希望从 0 到 $2m$ 的路径对应于清醒的工人（所接受的工作对应于他们的边缘，并且在工作之间路径沿着形式 $i \rightarrow i + 1$ 的边缘）。我们需要从这些路径中得到什么？他们必须涵盖所有复杂的工作。我们还有一些条件，例如“在分配清醒工人后，第 i 个时间段被不超过 b 个简单作业覆盖”。

但这个条件意味着第 i 个时隙的简单任务中，至少有 c_i 个完成。这意味着不超过 $a - c_i$ 的流量通过 $i \rightarrow i + 1$ 边。

换句话说，流动条件如下：

1. 一个简单任务的容量为 1。
2. 复杂任务恰好有 1 个单位的流量通过边缘。
3. 边 $i \rightarrow i + 1$ 的容量等于 $a - c_i$ 。
4. 总流量为 a 。

这是一个 l-r 流，图和流的大小为 $O(m)$ 。时间复杂度为 $O(m^2)$ 。